

Study 프로젝트 — 백엔드 데이터 흐름 정리

Spring Boot 4.0.6 · Java 17 · Spring Data JPA + MyBatis · MariaDB · Spring Security Crypto · springdoc-openapi · Spring Mail
본 문서는 `src/main/java/com/example/study` 패키지의 코드 기준으로 정리된 백엔드 데이터 흐름 안내입니다.

목차

- 전체 아키텍처 개요
- 요청 처리 계층 (Controller → Service → Repository → DB)
- 도메인별 데이터 흐름
 - 인증 / 회원 (로컬 + 카카오 / 네이버 / 구글 OAuth)
 - 이메일 인증 · 비밀번호 재설정
 - 게시판 (Notice / Comment / Like)
 - 카카오페이 단건 결제
 - 외부 API 프록시 (장소·도서·날씨·미세먼지·주식)
 - 관리자 메일 / 일일 발송 스케줄러
- 데이터베이스 모델
- 횡단 관심사 (CORS · 세션 · 예외 처리 · OpenAPI)

1. 전체 아키텍처 개요

요청은 항상 다음 4계층을 통과합니다.

Client (React@5173) → Controller (@RestController, /api/) → Service (@Service, @Transactional) → Repository (JpaRepository) / RestClient (외부 API) → MariaDB / 외부 API**

- 진입점:** `StudyApplication` — `@SpringBootApplication` + `@EnableAsync` + `@EnableScheduling`.
- CORS:** `WebConfig` — `/api/**` 에 대해 `localhost:3000/5173/5174/5175` 허용, `allowCredentials=true` (세션 쿠키 전달).
- 세션 인증:** 모든 보호된 컨트롤러는 `HttpSession` 의 `AuthController.SESSION_USER_KEY = "LOGIN_USER_ID"` 값을 검사. 토큰 기반이 아닌 **쿠키 + JSESSIONID** 세션 방식.
- 영속성:** 게시판/유저/결제 등 주요 도메인은 JPA(`spring.jpa.hibernate.ddl-auto=update`). MyBatis 도 스타터로 포함되어 있어 `classpath:mapper/*.xml` 사용 가능(현재 활성 매퍼 없음).
- 외부 호출:** 도서/장소/카카오페이/카카오톡/날씨/대기/주식/메일 모두 `RestClient` 또는 `JavaMailSender` 사용.

2. 요청 처리 계층

계층	대표 클래스	역할
Controller	<code>AuthController</code> · <code>NoticeController</code> · <code>CommentController</code> · <code>PaymentController</code> · <code>PlaceApiController</code> · ...	URL 매핑(<code>/api/**</code>), 세션 인증 확인, DTO 검증(<code>@Valid</code>), 결과를 <code>ResponseEntity</code> / DTO 로 반환.
DTO	<code>request/*</code> (<code>LoginRequest</code> , <code>SignupRequest</code> , <code>KakaoPayReadyRequest</code> ...) · <code>response/*</code> (<code>UserResponse</code> , <code>NoticeDetailResponse</code> , <code>MessageResponse</code> ...)	HTTP 입출력 전용 객체. 엔티티가 직접 노출되지 않도록 분리. <code>@JsonProperty</code> 로 스네이크/카멜 매핑.
Service	<code>AuthService</code> · <code>NoticeService</code> · <code>KakaoPayService</code> ·	비즈니스 로직 + 트랜잭션 경계(<code>@Transactional</code>). 외부 API 호출, BCrypt 해시, OAuth 토큰 교환 등.

	EmailVerificationService · PlaceService · DailyMailScheduler ...	
Repository	UserRepository · NoticeRepository · CommentRepository · NoticeLikeRepository · CommentLikeRepository · PaymentRepository · EmailVerificationRepository	<code>JpaRepository<T, ID></code> 기본 메서드 + 명명 메서드. 일부에 <code>@Query</code> 로 카운트/집계 쿼리.
Entity (DB 테이블)	User · Notice · Comment · NoticeLike · CommentLike · Payment · EmailVerification	JPA 매핑. <code>@PrePersist</code> / <code>@PreUpdate</code> 로 created_at/updated_at 자동 채움.
예외 처리	GlobalExceptionHandler	<code>@RestControllerAdvice</code> . <code>IllegalArgumentException</code> →404, <code>Validation</code> →400, 그 외→500. 모두 <code>MessageResponse</code> 로 변환.

HTTP 메서드 색상: GET POST PUT DELETE 태그: 외부API DB쓰기 인증필요

3. 도메인별 데이터 흐름

3-1. 인증 / 회원 가입 · 로그인

① 로컬 회원가입 (이메일 인증 선행)

POST /api/auth/email/send-code → EmailVerificationService.sendCode() → 6자리 코드 생성 → email_verification 테이블 upsert → JavaMailSender 로 SMTP(네이버/Gmail) 전송
POST /api/auth/email/verify-code → 코드 일치 + 만료 검사 → verified=true 로 표시
POST /api/auth/signup → UserAccountController → AuthService.signup() → username 중복 + 이메일 인증 확인 → BCrypt 패스워드 해시 → users 저장 → 인증 코드 consume → EmailService.sendWelcome() → 세션에 LOGIN_USER_ID 저장 후 UserResponse 반환.

② 로컬 로그인 / 로그아웃 / 내 정보

POST /api/auth/login → AuthService.login() : username 조회 + passwordEncoder.matches() → 성공 시 세션에 user.id 저장 / 실패 시 401.
POST /api/auth/logout → session.invalidate() .
GET /api/auth/me → 세션의 user.id 로 UserRepository.findById() → UserResponse (없으면 401).

③ 소셜 로그인 — 카카오 / 네이버 / 구글 OAuth 2.0

세 공급자 모두 동일한 패턴이며, 네이버/구글은 CSRF 방지를 위한 state 파라미터를 사용합니다.

예: 카카오

GET /api/auth/kakao/login?returnTo=... → 세션에 RETURN_TO 저장 → KakaoAuthService.authorizeUrl() 로 302 redirect.
카카오 인증 완료 후 카카오가 콜백 호출:
GET /api/auth/kakao/callback?code=... → 동일 code 재사용 차단(KAKAO_LAST_CODE) → AuthService.syncKakaoUser(code)
↳ KakaoAuthService.exchangeCode() → 외부API kauth.kakao.com/oauth/token
↳ fetchUser(accessToken) → 외부API kapi.kakao.com/v2/user/me
↳ users 에 kakao_id 로 upsert + 토큰/만료/리프레시 저장 → 세션 부여 → returnTo 로 redirect.

네이버는 SESSION_NAVER_STATE , 구글은 SESSION_GOOGLE_STATE 로 state 검증 후 동일 흐름. 실패 시 ? auth_error=...&auth_error_description=... 쿼리스트링으로 프론트로 돌려보냅니다.

3-2. 비밀번호 찾기 / 재설정 / 프로필 수정

POST /api/auth/password/forgot → AuthService.issueResetToken() : 32바이트 랜덤 토큰 생성 → users.reset_token + 30분 만료 저장 → EmailService.sendPasswordReset() 로 {frontendUrl}/reset?token=... 안내.
POST /api/auth/password/reset → 토큰 + 만료 검증 → 새 비밀번호 BCrypt 해시 저장.
PUT /api/auth/me 인증필요 → 닉네임/이메일/비밀번호/알림수신 변경. 소셜 전용 계정은 이메일/비밀번호 변경 차단 (SOCIAL_LOCKED).

3-3. 게시판 — Notice · Comment · Like

엔드포인트	흐름
GET /api/notices	NoticeService.search(type,keyword,Pageable) → 제목/본문 LIKE 검색 + 페이지 네이션 → enrich() 에서 댓글 수·좋아요 수를 batch 조회해 NoticeListItem 으로 매핑.
GET /api/notices/{id}	findDetail → Notice + 댓글 수 + 좋아요 수 + 현재 사용자의 좋아요 여부 → NoticeDetailResponse .

POST /api/notices/{id}/views	DB쓰기 notice.view_count++ .
POST /api/notices/{id}/like 인증필요	NoticeLikeService.toggle() : notice_like 행 존재 여부로 추가/삭제 토글, 카운트는 집계 쿼리.
GET /api/notices/{id}/likes	좋아요 누른 사용자 목록 → LikedUserResponse .
POST /api/notices	본문/제목 검증(@Valid) → Notice 신규 생성, authorId = currentUserId .
PUT /api/notices/{id}	기존 글 수정 (없으면 새 Notice 객체에 세팅 후 save).
DELETE /api/notices/{id}	존재 확인 후 삭제, 없으면 IllegalArgumentException → 전역 핸들러가 404.
GET /api/notices/{id}/comments	CommentService.listByNoticeId(id, me) → 좋아요 카운트와 함께 CommentResponse 리스트.
POST /api/notices/{id}/comments 인증필요	댓글 신규 작성. 비로그인 401.
PUT /api/comments/{commentId}	본인 댓글만 수정 가능. 본인 아니면 SecurityException → 403.
DELETE /api/comments/{commentId}	본인 댓글만 삭제. 403 동일.
POST /api/comments/{commentId}/like 인증필요	CommentLikeService.toggle() .

3-4. 카카오페이 단건 결제 (TC00NETIME)

① **ready** POST /api/payments/kakao/ready
Body: { itemName, quantity, totalAmount }
→ KakaoPayService.ready() :
↳ partnerOrderId (UUID 12자) · partnerUserId 생성
↳ Payment 엔티티 신규 저장 (status=READY) DB쓰기
↳ 외부API POST open-api.kakaopay.com/online/v1/payment/ready (Authorization: SECRET_KEY \${kakaopay.secret-key})
↳ 응답에서 tid, PC/모바일/앱 결제창 URL 추출 → tid 를 Payment 에 저장
↳ KakaoPayReadyResponse 로 결제창 URL 들 반환 → 프론트가 next_redirect_pc_url 로 이동.

② **approve** (사용자 결제 완료 후 카카오가 콜백) GET /api/payments/kakao/approve?orderId=...&pg_token=...
→ KakaoPayService.approve() : partnerOrderId 로 Payment 조회 → tid/pgToken 으로 외부API /online/v1/payment/approve 호출 → 응답의 aid, payment_method_type 저장, status=APPROVED, approvedAt=now DB쓰기 → 프론트의 /pay/success?orderId&amount&itemName 로 302 redirect.

③ **cancel / fail** → Payment.status 를 CANCELED / FAILED 로 변경 후 프론트 /pay/fail?reason=... 로 redirect.

④ **내 결제 내역** GET /api/payments/kakao/my 인증필요 → paymentRepository.findByUserIdOrderByCreatedAtDesc() .

3-5. 외부 API 프록시

프론트가 카카오/공공 API 키를 노출하지 않도록 백엔드가 단순 프록시 + 정규화 + 풀백을 담당합니다.

엔드포인트	업스트림	요점
GET /api/places/search?query&page&size	Kakao v2/local/search/keyword.json	Kakao REST API 키워드 검색. page 1~45, size 1~15 클램프.
GET /api/places/nearby?lat&lng&category&radius&size	카테고리 API (기본 FD6 음식점) → 쿼터 초과 시 키워드 검색으로 자동 풀백 (커밋 0212ab1)	500 대신 빈 결과(EMPTY) 반환. 응답 필드는 @JsonProperty 로 카멜케이스 매핑.
GET /api/books/search?query&sort&target&page&size	Kakao v3/search/book	도서 검색.
GET /api/weather?lat&lng	기상청/외부 날씨 API	좌표 → 현재 날씨. WeatherResponse 로 정규화.
GET /api/air?lat&lng	대기질 API	미세먼지/초미세먼지 + 등급.
GET /api/stocks/top-gainers?limit	주식 API	큐레이션 종목 중 상승률 TOP N.
GET /api/stocks/top-losers?limit	동일	하락률 TOP N.

3-6. 관리자 메일 / 일일 발송 스케줄러

관리자 식별: app.admin.username (기본 test11) 과 로그인 user.username 일치 시. 또는 헤더 X-Admin-Token 이 app.admin.token 과 일치 시 우회 허용(개발/스크립트용).

GET /api/admin/me → admin 여부 + 수신동의 구독자 수.
POST /api/admin/broadcast → 동의자 전원에게 HTML 메일 발송 (EmailService.sendBroadcast).
POST /api/admin/broadcast/daily-now?force=true → DailyMailScheduler.forceSendNow() 강제 실행.

스케줄: DailyMailScheduler 는 @EnableScheduling 하에 매일 KST 20:00 실행.

→ 수신동의 사용자 조회 → 서울 좌표 기준 날씨/대기/주식/뉴스 수집(`WeatherService` , `AirQualityService` , `StockService` , `NewsService`) → HTML 본문 작성 → `EmailService` 로 발송
→ 추가로 `KakaoTalkMessageService` 가 "나에게 보내기" API로 카카오톡 메시지 발송 (수신자가 카카오 토큰을 보유 + opt-in)
→ 발송 완료 일자를 `~/study-notice-last-daily.txt` 에 기록(중복 발송 방지).

4. 데이터베이스 모델

`spring.jpa.hibernate.ddl-auto=update` 설정으로 엔티티 변경 시 자동 마이그레이션. 데이터베이스는 `jdbc:mariadb://localhost:3306/notice`.

테이블	주요 컬럼	비고
users	id PK · username (uk) · password (BCrypt) · nickname · email · profile_image · kakao_id (uk) · naver_id (uk) · google_id (uk) · kakao_access_token · kakao_refresh_token · kakao_token_expires_at · kakao_talk_opt_in · reset_token · reset_token_expires_at · notification_opt_in · created_at · updated_at	로컬 + 3사 소셜이 같은 row 에 누적. 카카오 토큰은 카카오톡 발송용으로 저장.
notice	id · author · author_id · title · content (TEXT) · view_count · created_at	JPA + Lombok. <code>@PrePersist</code> 로 created_at 자동.
notice_comment	id · notice_id · user_id · content (≤2000) · created_at · updated_at	인덱스 (<code>notice_id</code> , <code>created_at</code>) 로 게시물 별 정렬.
notice_like	id · notice_id · user_id · created_at	한 사용자가 한 게시물에 최대 1행. count/exists 쿼리로 토글.
comment_like	id · comment_id · user_id · created_at	댓글 좋아요. 동일 토글 패턴.
payment	id · partner_order_id · partner_user_id · user_id · item_name · quantity · total_amount · tid · aid · payment_method_type · status (READY/APPROVED/CANCELED/FAILED) · approved_at · created_at	카카오페이 단건결제 트랜잭션 한 건당 1 row.
email_verification	이메일 · 6자리 코드 · expires_at · verified · 발송 횟수	회원가입 사전 이메일 인증. <code>consume()</code> 시 제거.

5. 횡단 관심사

5-1. 인증 & 세션

- 모든 컨트롤러는 `HttpSession.getAttribute(AuthController.SESSION_USER_KEY)` 로 현재 사용자 ID를 가져옵니다.
- OAuth 콜백은 `RETURN_TO`, 상태값(`NAVER_STATE`, `GOOGLE_STATE`), 마지막 code(`*_LAST_CODE`) 를 세션에 임시 보관 → 재사용/위변조 방지.
- 쿠키: `same-site=lax`, `http-only=true`.
- 비밀번호: `BCryptPasswordEncoder` (spring-security-crypto). 재설정 토큰은 `SecureRandom` + URL-safe Base64.

5-2. CORS

- `/api/**` 에 대해 React 개발 포트(3000/5173/5174/5175) 만 허용. `allowCredentials=true` 로 세션 쿠키 사용.

5-3. 예외 처리 (`GlobalExceptionHandler`)

예외	HTTP 상태	응답
<code>IllegalArgumentException</code> (도메인에서 의도적으로 던짐)	404 NOT_FOUND	<code>MessageResponse</code> with 메시지
<code>MethodArgumentNotValidException</code> (@Valid 실패)	400	필드별 메시지 줄바꿈 합치기
<code>MethodArgumentTypeMismatch</code> / <code>MissingServletRequestParameter</code>	400	"잘못된 요청 파라미터: ..."

그 외 Exception	500	ClassName: message
SecurityException (CommentService)	403 (컨트롤러에서 직접 처리)	메시지 반환

5-4. OpenAPI · 문서화

- `springdoc-openapi-starter-webmvc-ui` 포함 → 기본적으로 `/swagger-ui/index.html`, `/v3/api-docs` 가 활성화됩니다.
- `OpenApiConfig` 는 OpenAPI 메타데이터(타이틀/설명/버전) 를 정의.

5-5. 비동기 / 스케줄링

- `@EnableAsync` 로 메일 등 일부 작업은 비동기 처리 가능.
- `@EnableScheduling` 로 `DailyMailScheduler.@Scheduled` 가 매일 KST 20:00 동작.

5-6. 외부 API 키 관리

- Kakao REST API 키: `kakao.api.key`
- Kakao OAuth: `kakao.oauth.client-id/secret/redirect-uri`
- Naver / Google OAuth: 환경변수 `${NAVER_CLIENT_ID}`, `${GOOGLE_CLIENT_ID}` ... 주입.
- Kakao Pay: `kakaopay.cid` (테스트: TC0ONETIME), `kakaopay.secret-key` (환경변수 `KAKAOPAY_SECRET_KEY` 권장).
- SMTP: 기본 `smtp.naver.com:465`, `MAIL_HOST` 등 환경변수로 Gmail 전환 가능.
- 관리자: `app.admin.username` (기본 `test11`), `app.admin.token` (헤더 우회용).

끝. 본 문서는 자동 생성된 정적 스냅샷이며, 코드 변경 시 함께 갱신하세요.